

Sistemas de Percepción y Visión Artificial

3.^o Grado en Ingeniería en la Industria Conectada

Sistema de Detección y Medición de Defectos en Baldosas Cerámicas mediante Deep Learning

Modelo: YOLOv8m-seg · Segmentación de Instancias

Pipeline: Adquisición → Etiquetado → Pseudo-labeling
→ Entrenamiento → Inferencia → Aplicación Web

Autor: Javier Barba Berrocal

Fecha: 14 de mayo de 2026

Repositorio: <https://github.com/jaaviixoo06>

Resumen

El presente trabajo documenta el desarrollo íntegro de un sistema de visión artificial orientado a la detección y cuantificación automática de defectos superficiales en baldosas cerámicas industriales. La tipología de defectos contemplada abarca dos categorías fundamentales: rayaduras —incisiones lineales o curvas sobre el esmalte— y agujeros o desportillados —pérdidas puntuales de material—.

El pipeline construido comprende cinco fases integradas: adquisición y selección de imágenes; preprocesado, etiquetado manual con Label Studio y etiquetado automático por pseudo-labeling; entrenamiento de la red YOLOv8m-seg en GPU mediante la plataforma Kaggle; un módulo de inferencia capaz de calcular la longitud, grosor y área de cada defecto en milímetros reales; y una aplicación web interactiva construida con FastAPI y HTML/JavaScript que permite inspeccionar cualquier imagen desde el navegador y exportar el informe de resultados.

Los resultados sobre el conjunto de validación arrojan un Box mAP@0.5 = 0.806 y Mask mAP@0.5 = 0.520, con tiempos de inferencia de aproximadamente 480 ms en CPU. El dataset final de entrenamiento consta de 2978 imágenes (2532 train + 446 val), construido a partir de 148 anotaciones manuales ampliadas mediante pseudo-labeling y aumentos con Albumentations.

Índice

1	Introducción y Motivación	4
1.1	Objetivos del proyecto	4
1.2	Clases de defecto	4
2	Fase 1 — Adquisición y Selección de Imágenes	4
2.1	Contexto: fases de prueba PRUEBA1–PRUEBA5	4
2.2	Criterios de selección	5
2.3	Datasets externos evaluados	5
3	Fase 2 — Tratamiento de Imágenes y Construcción del Dataset	6
3.1	Evolución del aprendizaje en filtros: de las monedas a las baldosas	6
3.1.1	Primera etapa: detección de bordes sobre monedas	6
3.1.2	Segunda etapa: inpainting y Difference of Gaussians sobre imagen de carretera	8
3.1.3	Tercera etapa: pipeline completo sobre baldosas cerámicas	9
3.2	Filtros de preprocesado evaluados para el modelo	10
3.2.1	Umbralización binaria (Otsu)	10
3.2.2	Suavizado gaussiano	11
3.2.3	CLAHE	11
3.2.4	Blackhat morfológico	11
3.2.5	Resumen de decisiones de preprocesado	12
3.3	Herramienta de etiquetado: Label Studio	12
3.4	Formato de etiquetas YOLO-seg	15
3.5	Pseudo-labeling: ampliación semi-supervisada del dataset	16
3.5.1	Algoritmo	16
3.5.2	Resultados por ronda	17
3.6	Aumento de datos	17
3.7	Composición final del dataset	18
4	Fase 3 — Entrenamiento del Modelo	18
4.1	La decisión de consolidar clases: del fracaso de tres a la solidez de dos	18
4.2	Arquitectura: YOLOv8m-seg	19
4.3	Historia de las plataformas de entrenamiento	19
4.3.1	Primer intento: CPU local (Intel Core i7, 14 horas)	19
4.3.2	Segundo intento: Google Colab (interrumpido en la época 140/150)	20
4.3.3	Solución definitiva: Kaggle Notebooks (150 épocas en 3,2 horas)	20
4.4	Hiperparámetros de entrenamiento	21
4.5	Progresión del entrenamiento	22
4.6	Resultado del entrenamiento	22
5	Fase 4 — Módulo de Inferencia	22
5.1	Pipeline de inferencia	22
5.2	Medición de defectos	23
5.3	Calibración de escala px→mm	23
5.4	Ejemplo de salida de inferencia	24
6	Fase 5 — Aplicación Web: VISION_TILE_CORE	24

6.1	Propuesta de valor: un problema industrial real	24
6.2	La fórmula de medición de defectos	24
6.2.1	Rectángulo de área mínima (<code>cv2.minAreaRect</code>)	25
6.2.2	Conversión a milímetros reales	25
6.3	Página principal: punto de entrada al sistema	26
6.4	Animación de análisis: retroalimentación en tiempo real	27
6.5	Dashboard de resultados: el núcleo del sistema	27
6.6	Lightbox de defecto individual	28
6.7	Tabla de medidas: trazabilidad industrial	29
6.8	API REST: integración con sistemas externos	29
7	Resultados y Evaluación	30
7.1	Evolución de métricas a lo largo del proyecto	30
7.2	Análisis de errores	30
7.3	Rendimiento computacional	30
8	Conclusiones y Trabajo Futuro	30
8.1	Conclusiones	30
8.2	Trabajo futuro	31
	Referencias	31
	Apéndice A — Estructura del proyecto	32
	Apéndice B — Imágenes incluidas	35

1 Introducción y Motivación

La industria cerámica requiere un control de calidad exhaustivo de sus productos antes de su comercialización. Es relevante señalar que las baldosas cerámicas pueden presentar defectos generados tanto durante el proceso de fabricación como durante la manipulación y el transporte: **rayaduras** —incisiones lineales o curvas que comprometen la integridad superficial del esmalte— y **agujeros o desportillados** —pérdidas localizadas de material que generan huecos de profundidad variable—.

La inspección visual humana, aun siendo el método tradicional, adolece de limitaciones inherentes: es lenta, subjetiva y extraordinariamente propensa a la fatiga del operario. En este contexto, la visión artificial emerge como la herramienta paradigmática para automatizar este proceso con mayor repetibilidad, velocidad y trazabilidad. Resulta imprescindible, por tanto, explorar las posibilidades que ofrece el aprendizaje profundo aplicado a imágenes de dominio industrial específico.

1.1 Objetivos del proyecto

1. Construir un dataset propio de imágenes de baldosas cerámicas con defectos reales, anotadas a nivel de *segmentación de instancias*.
2. Entrenar un modelo de detección y segmentación (YOLOv8m-seg) capaz de localizar y contornear cada defecto con alta confianza.
3. Implementar un módulo de medición que calcule la **longitud, grosor y área** de cada defecto en **milímetros reales** a partir de una única imagen.
4. Desplegar el sistema en una **aplicación web** funcional, accesible desde cualquier navegador sin instalación de software adicional.

1.2 Clases de defecto

Se han definido dos clases de defecto alineadas con la nomenclatura industrial. Cabe mencionar que esta elección —aparentemente sencilla— es en sí misma el resultado de una iteración experimental que se detalla en la Sección 4.1:

ID	Nombre	Descripción
0	agujero	Desportillado puntual; pérdida de material en el esmalte
1	rayadura	Incisión lineal o curva en la superficie de la baldosa

2 Fase 1 — Adquisición y Selección de Imágenes

2.1 Contexto: fases de prueba PRUEBA1–PRUEBA5

El proceso de adquisición de datos no fue lineal, sino progresivo e iterativo. Lejos de partir de un conjunto de imágenes definitivo, se organizó el trabajo en cinco fases de exploración —almacenadas bajo el directorio `testing/pruebas_detector/` del repositorio— que permitieron comprender el dominio visual del problema antes de abordar el etiquetado a gran escala.

- PRUEBA1** Primeras capturas fotográficas de baldosas del entorno del laboratorio. El objetivo no era aún construir un dataset, sino entender la variabilidad intrínseca del problema: resolución, iluminación, ángulo de captura y el grado de contraste de los defectos frente al esmalte.
- PRUEBA2** Ampliación del número de muestras con la incorporación de baldosas que presentaban defectos de mayor envergadura. En esta fase se evaluó la calidad mínima de imagen requerida para que el etiquetado posterior fuera viable.
- PRUEBA3** Primeras pruebas de anotación manual con herramientas básicas de dibujo de polígonos. Se detectaron dificultades específicas para anotar rayaduras finas de anchura inferior a 2 mm, cuyo contraste resulta insuficiente sin preprocesado.
- PRUEBA4** Integración de Label Studio como plataforma de etiquetado profesional. En esta fase se definió el esquema de clases definitivo y se estableció el formato de exportación YOLO-seg como estándar del proyecto.
- PRUEBA5** Fase de producción: directorio `base_images` con **2 690 imágenes** de alta calidad de baldosas cerámicas reales. Este directorio constituye la fuente principal tanto para el entrenamiento como para el pseudo-labeling posterior.

2.2 Criterios de selección

Para garantizar la calidad del dataset se establecieron los siguientes criterios de admisión de imágenes, que actuaron como filtro previo al etiquetado:

- La baldosa debe ocupar al menos el 60 % del encuadre de la imagen.
- Resolución mínima de 640×640 píxeles.
- Iluminación uniforme, sin reflejos especulares que enmascaren los defectos.
- Presencia de al menos un defecto visible por imagen en el subconjunto de etiquetado manual.

2.3 Datasets externos evaluados

Asimismo, se evaluaron dos fuentes externas de datos como potenciales complementos al dataset propio:

Dataset	Descripción	Imágenes	Adoptado
DATN Ceramic Tile	Baldosas cerámicas reales con defectos etiquetados	~800	Sí
DBCC Bridge Crack	Grietas en estructuras de hormigón y puentes	~32 000	No

El dataset DBCC fue descartado a pesar de contener imágenes de grietas lineales morfológicamente similares a las rayaduras. La razón determinante fue el *domain shift*: el dominio visual de una superficie gris rugosa de hormigón dista considerablemente del esmalte cerámico liso y brillante, hasta el punto de que entrenar con imágenes de hormigón introducía falsos positivos sistemáticos en texturas de fondo de las baldosas. En consecuencia, se optó por privilegiar la pureza del dominio sobre la cantidad bruta de datos.

3 Fase 2 — Tratamiento de Imágenes y Construcción del Dataset

3.1 Evolución del aprendizaje en filtros: de las monedas a las baldosas

Es relevante subrayar que el desarrollo del preprocesado de imágenes no partió directamente de las baldosas cerámicas, sino de una progresión pedagógica que permitió comprender el comportamiento de los filtros en contextos de creciente complejidad. Esta metodología —de lo simple a lo complejo— resultó determinante para tomar decisiones informadas sobre el preprocesado final del sistema.

3.1.1 Primera etapa: detección de bordes sobre monedas

El primer ejercicio de procesamiento de imagen se realizó sobre un conjunto de **monedas de distintos tamaños y relieves**, que ofrecen un entorno controlado para evaluar detectores de bordes clásicos: Canny, Sobel y Laplaciano.

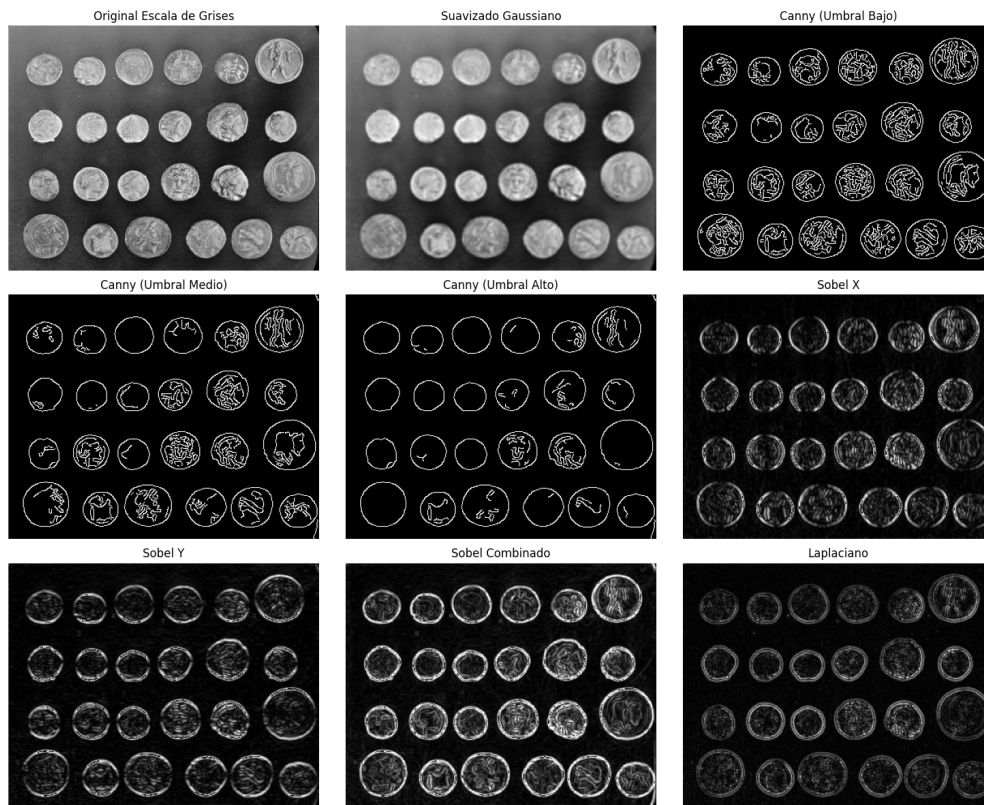


Figura 1: Comparativa de detectores de borde aplicados sobre monedas. De izquierda a derecha y de arriba a abajo: imagen original en escala de grises, suavizado gaussiano previo, Canny con umbral bajo (muchos bordes, ruido), Canny con umbral medio (equilibrado), Canny con umbral alto (solo bordes principales), Sobel en dirección X, Sobel en Y, Sobel combinado y Laplaciano. Esta comparativa permitió comprender la sensibilidad de cada operador frente al ruido y su capacidad de resaltar estructuras de diferente escala.

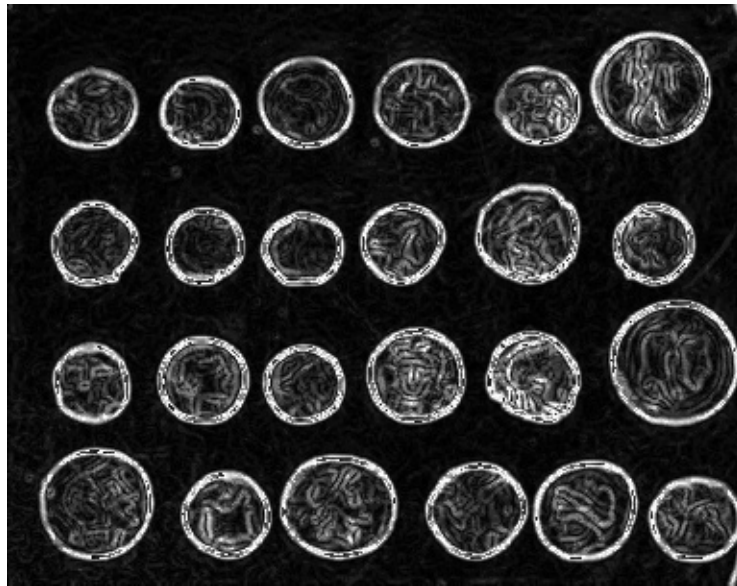


Figura 2: Resultado del filtro Sobel combinado ($\sqrt{G_x^2 + G_y^2}$) sobre el conjunto completo de monedas. La magnitud del gradiente resalta con claridad los contornos circulares y los relieves internos de cada moneda, aunque presenta mayor sensibilidad al ruido superficial que el operador Canny con umbral medio.

De este ejercicio se extrajeron conclusiones fundamentales: el operador Canny con umbral medio ofrece el mejor compromiso entre sensibilidad y robustez frente al ruido; el Laplaciano, en cambio, amplifica el ruido de sensor de forma inaceptable en imágenes de bajo contraste. No obstante, en el contexto de las baldosas cerámicas —cuyas rayaduras presentan un perfil de contraste radicalmente diferente al de los relieves de una moneda—, estos operadores de segundo orden demostraron ser insuficientes por sí solos.

3.1.2 Segunda etapa: inpainting y Difference of Gaussians sobre imagen de carretera

El segundo ejercicio abordó una problemática de mayor complejidad: la detección de marcas viales sobre una **imagen de carretera nocturna**. En este caso, la dificultad no residía en la detección de bordes, sino en la eliminación del ruido de fondo (la textura del asfalto) para aislar únicamente las marcas de interés.

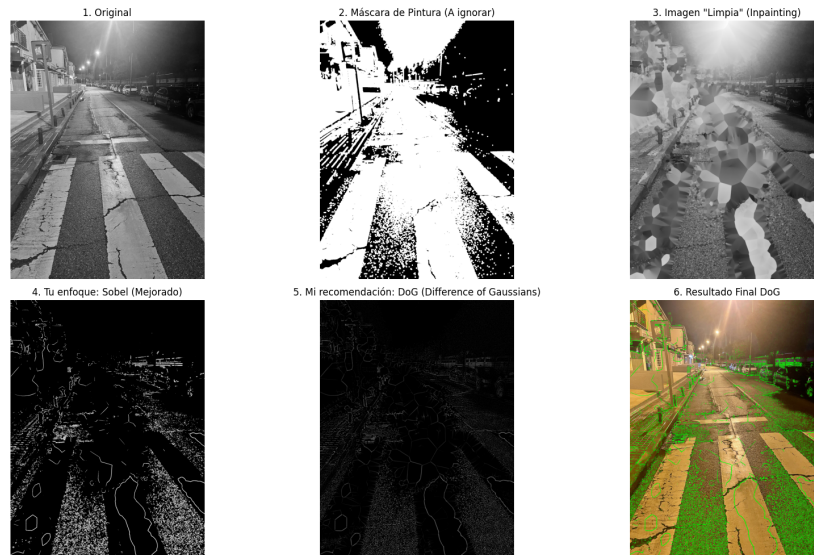


Figura 3: Comparativa de dos estrategias sobre imagen de carretera. Arriba: imagen original, máscara de pintura (zonas a ignorar) e imagen limpia obtenida mediante *inpainting*. Abajo: el enfoque inicial con Sobel (mejorado), la propuesta final con *Difference of Gaussians* (DoG) y el resultado final. El DoG suprime el ruido de baja frecuencia de la textura del asfalto mientras preserva las estructuras finas de las marcas, demostrando ser significativamente superior al operador Sobel para este tipo de problema.

El operador **Difference of Gaussians** (DoG) —diferencia entre dos imágenes suavizadas con kernels gaussianos de distinto sigma— actúa como un filtro paso-banda espacial: elimina tanto las frecuencias muy bajas (fondo uniforme) como las muy altas (ruido de sensor), preservando las estructuras de tamaño intermedio. Este principio, aprendido en el ejercicio de la carretera, resultó clave para orientar la elección del filtro *blackhat* morfológico en la aplicación final sobre baldosas.

3.1.3 Tercera etapa: pipeline completo sobre baldosas cerámicas

La aplicación directa de los aprendizajes anteriores al dominio objetivo condujo al diseño de un pipeline de preprocesado específico para la detección de rayaduras en baldosas. En este caso, la característica diferencial de las rayaduras —estructuras oscuras y alargadas sobre un fondo claro y uniforme— hacía que el operador **blackhat morfológico** fuera la herramienta más adecuada.

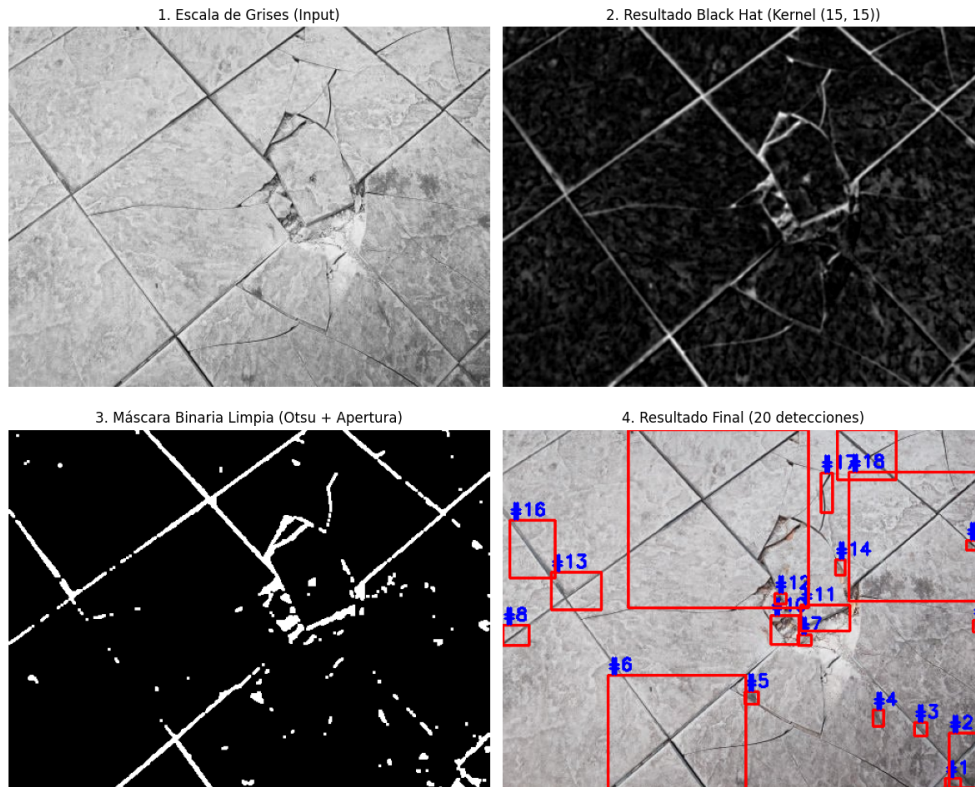


Figura 4: Pipeline de preprocesado y detección sobre imagen de baldosa. (1) Imagen original en escala de grises; (2) resultado del filtro Black Hat con kernel 15×15 , que realiza las rayaduras y juntas como estructuras oscuras sobre fondo claro; (3) máscara binaria obtenida tras umbralización de Otsu y operación morfológica de apertura, que elimina el ruido residual; (4) resultado final con 20 detecciones etiquetadas. Este pipeline sirvió como referencia para el diseño de la segunda pasada de verificación en el módulo de inferencia.

3.2 Filtros de preprocesado evaluados para el modelo

A la luz de la progresión anterior, se evaluaron cuatro técnicas de preprocesado específicamente sobre el dataset de baldosas, con el objetivo de determinar cuáles debían integrarse en el pipeline del modelo y cuáles debían descartarse.

3.2.1 Umbralización binaria (Otsu)

```

1 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
2 _, thresh = cv2.threshold(gray, 0, 255,
3                       cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

```

Listing 1: Umbralización de Otsu sobre canal de luminancia

La umbralización de Otsu calcula automáticamente el umbral óptimo minimizando la varianza intraclase entre fondo y objeto. Aplicada sobre la baldosa —cuya superficie es blanca y el fondo de la fotografía es oscuro—, el umbral resultante se sitúa en torno a 60 y segmenta correctamente la baldosa del fondo. No obstante,

no distingue entre la superficie esmaltada y las rayaduras, cuyo contraste local es insuficiente para esta operación global.

Decisión: adoptada exclusivamente para la calibración de escala px→mm en `detect_tile_px()`, no como preprocesado de entrada al modelo.

3.2.2 Suavizado gaussiano

```
1 blurred = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)
```

Listing 2: Suavizado gaussiano con kernel 5×5

El suavizado gaussiano elimina ruido de sensor, pero atenúa simultáneamente los bordes finos de las rayaduras. Para rayaduras de anchura inferior a 2 mm, el suavizado llega a hacerlas prácticamente invisibles en el espacio de 640 píxeles. Las pruebas empíricas con el modelo evidenciaron una pérdida de $\approx 1-2$ puntos de mAP en validación respecto a la imagen original sin filtro.

Decisión: descartado como preprocesado global; se mantiene como paso intermedio interno dentro de `detect_tile_px()` para reducir microestructuras del esmalte que interferirían con la detección del contorno de la baldosa.

3.2.3 CLAHE

```
1 lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
2 L, A, B = cv2.split(lab)
3 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
4 L_eq = clahe.apply(L)
5 img_clahe = cv2.cvtColor(cv2.merge([L_eq, A, B]), cv2.COLOR_LAB2BGR)
```

Listing 3: CLAHE en canal L del espacio de color LAB

La ecualización adaptativa de histograma (CLAHE) opera sobre la luminancia en el espacio LAB, mejorando el contraste local sin sobreexponer las zonas ya bien iluminadas. En baldosas con gradientes de iluminación, resalta las rayaduras en las zonas de menor exposición. Sin embargo, en condiciones de iluminación uniforme—predominantes en el dataset— el efecto es mínimo o introduce artefactos de halo en los bordes de la baldosa.

Decisión: incluido como transformación de aumentación en Albumentations con probabilidad 0.30, de modo que el modelo aprenda a generalizar ante variaciones de contraste, pero no aplicado de forma fija en inferencia.

3.2.4 Blackhat morfológico

```
1 kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15))
2 blackhat = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, kernel)
3 enhanced = cv2.addWeighted(gray, 1.0, blackhat, 2.5, 0)
```

Listing 4: Filtro blackhat para realzar rayaduras oscuras sobre fondo claro

El operador blackhat extrae las regiones más oscuras que su entorno local en un radio definido por el kernel. En el caso de las baldosas—esmalte blanco con rayaduras como incisiones oscuras—, con un kernel de 15×15 y un factor de amplificación

de 2.5 las rayaduras finas ganan entre 20 y 40 niveles de gris adicionales, haciéndolas visualmente inequívocas incluso en zonas de bajo contraste.

Decisión: integrado en `src/infer.py` como canal auxiliar de verificación. Cuando el modelo no detecta ningún defecto, se aplica blackhat como segunda pasada para confirmar que no existen rayaduras de contraste extremadamente bajo.

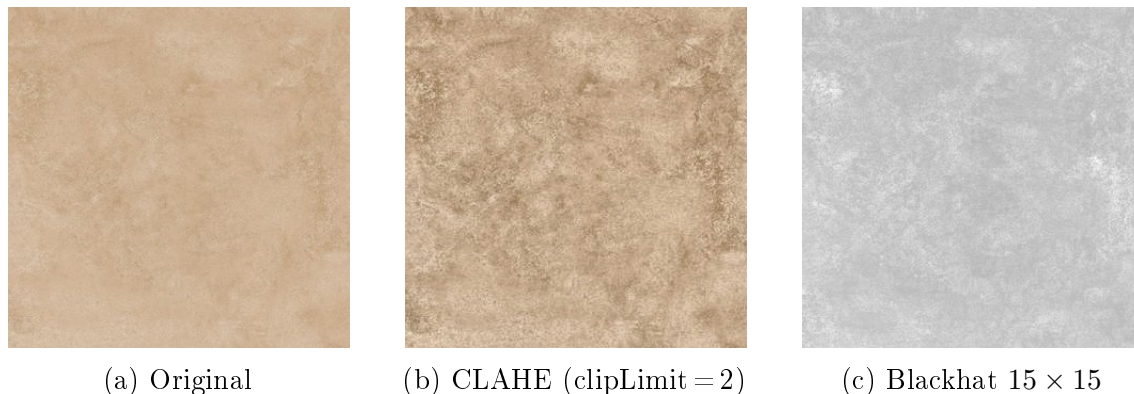


Figura 5: Comparativa de los tres filtros evaluados sobre un recorte de 256×256 px extraído de una imagen de baldosa real. CLAHE mejora el contraste global de forma apreciable, pero el operador blackhat es el único que hace la rayadura claramente distinguible del fondo, confirmando su idoneidad como canal auxiliar de detección.

3.2.5 Resumen de decisiones de preprocesado

Filtro	Rayaduras gruesas	Rayaduras finas	Agujeros	Integración
Sin filtro	Bueno	Regular	Bueno	Entrada principal al modelo
Threshold Otsu	Regular	Malo	Regular	Solo calibración de escala
Gaussiano 5×5	Regular	Malo	Regular	Descartado
CLAHE	Bueno	Regular	Bueno	Augmentación (prob.0.5)
Blackhat	Regular	Bueno	Malo	Segunda pasada post-inferencia

La conclusión principal de este análisis es que no existe un único filtro óptimo para todas las condiciones de defecto e iluminación. La estrategia adoptada —imagen original como entrada al modelo, blackhat como canal de verificación— resulta más robusta que cualquier preprocesado fijo de aplicación universal.

3.3 Herramienta de etiquetado: Label Studio

Se utilizó **Label Studio** (v1.x, modo local) como plataforma de anotación, seleccionada tras evaluar las principales alternativas disponibles. Esta herramienta permite etiquetar polígonos de segmentación directamente sobre la imagen en el navegador y exportar en múltiples formatos, entre ellos el formato YOLO-seg nativo.

La configuración empleada fue la siguiente:

- Tipo de tarea: *Image Segmentation* con polígonos de instancia.
- Clases: **agujero** (color verde) y **rayadura** (color naranja/rojo).
- Exportación: formato YOLO-seg (coordenadas normalizadas de polígono).

El proceso de etiquetado evolucionó a través de dos fases diferenciadas cuya distinción resulta determinante para comprender la calidad final del dataset.

Primera fase — bounding boxes para ambas clases. Tanto las rayaduras como los agujeros se etiquetaron inicialmente con rectángulos envolventes. Para los agujeros —formas compactas y redondeadas— este esquema resultó suficientemente preciso. Sin embargo, para las rayaduras —trazos curvos y delgados de anchura de uno a tres píxeles— la caja envolvente capturaba una proporción desproporcionada de esmalte intacto, contaminando las máscaras y degradando la capacidad discriminativa del modelo.

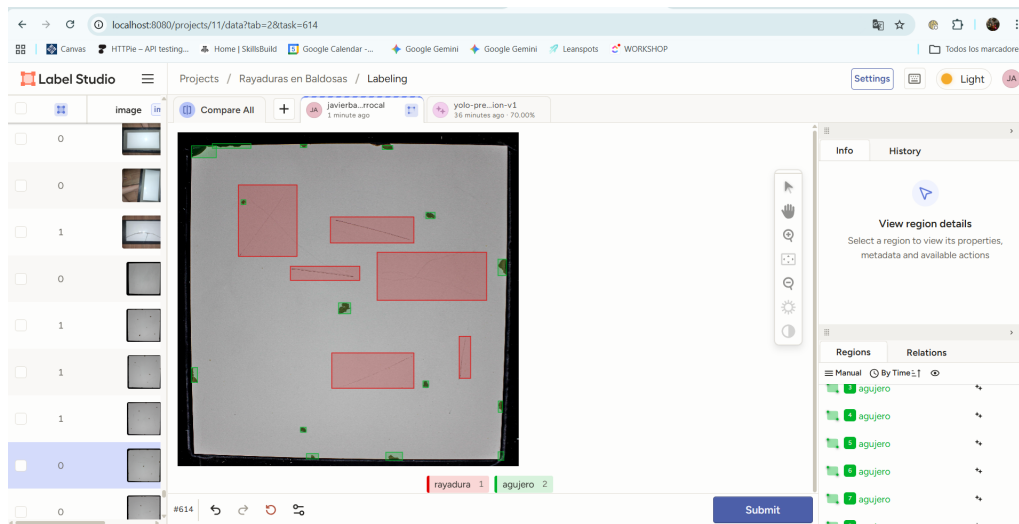


Figura 6: Label Studio durante la **primera fase** de etiquetado: ambas clases —rayadura y agujero— se anotan con bounding boxes rectangulares. Los rectángulos rosas delimitan las regiones marcadas sobre la baldosa. Para los agujeros, de forma compacta, esta representación resultó válida; para las rayaduras, la caja envolvente incluía demasiado esmalte intacto, lo que degradaba la calidad de las máscaras de segmentación y motivó la transición al etiquetado poligonal.

Segunda fase — polígonos de precisión para rayaduras. Reconocida esta limitación, se migró al etiquetado poligonal exclusivamente para la clase **rayadura**, trazando el contorno real de cada incisión vértice a vértice. Los agujeros mantuvieron la representación rectangular, suficiente para su forma compacta. Esta decisión elevó el coste de anotación por imagen pero se tradujo en una ganancia sustancial en la precisión de las máscaras y, por extensión, en la calidad de las mediciones métricas calculadas en inferencia.

El proceso de etiquetado manual resultó en **148 imágenes** anotadas, con un total estimado de 320 instancias de defecto entre ambas clases.



Figura 8: Resultado de inferencia sobre una baldosa real tras la segunda fase de etiquetado. Las máscaras rojas ciñen con precisión cada rayadura y los contornos verdes delimitan con exactitud los agujeros y desportillados. El salto cualitativo respecto a la primera fase —en la que los bounding boxes incluían fondo— es apreciable en la nitidez de los contornos de segmentación.

Justificación de la elección de Label Studio

Herramienta	Tipo de anotación	Export YOLO-seg	Gratuita
Roboflow	Polígono, caja, keypoint	Sí (con cuenta)	Parcial
CVAT	Polígono completo	Sí (convertido)	Sí
Label Studio	Polígono completo	Sí nativo	Sí

Label Studio fue elegido por su exportación directa al formato YOLO-seg sin conversión intermedia, su instalación local sin límite de imágenes y su capacidad de usar el mismo proyecto para comparar anotaciones manuales con predicciones del modelo en el flujo de pseudo-labeling.

3.4 Formato de etiquetas YOLO-seg

Cada fichero de etiqueta (.txt) contiene una línea por instancia de defecto:

`clase  $x_1$   $y_1$   $x_2$   $y_2$  ...  $x_n$   $y_n$`

donde `clase` $\in \{0, 1\}$ y las coordenadas están normalizadas respecto al ancho y alto de la imagen ($\in [0, 1]$). Este formato permite representar contornos arbitrariamente complejos con la precisión requerida para la segmentación de instancias.

3.5 Pseudo-labeling: ampliación semi-supervisada del dataset

Dado el elevado coste de la anotación manual —estimado en aproximadamente 8 minutos por imagen para rayaduras complejas—, se implementó un pipeline de **pseudo-labeling** que utiliza el propio modelo entrenado para generar etiquetas sobre las imágenes no anotadas del dataset. Esta técnica, enmarcada en el ámbito del aprendizaje semi-supervisado, permitió multiplicar el tamaño del dataset de forma controlada y verificable.

3.5.1 Algoritmo

1. Entrenar un modelo inicial M_0 con las 148 muestras manuales.
2. Aplicar M_0 sobre las 2 690 imágenes de `testing/pruebas_detector/PRUEBA5/base_images`.
3. Aceptar únicamente las predicciones con confianza $\geq 0,65$.
4. Combinar las etiquetas aceptadas con las manuales y reentrenar el modelo.
5. Repetir con el modelo mejorado M_1 para obtener pseudo-etiquetas de mayor calidad.

Nota técnica

El umbral de 0.65 es crítico. Por debajo de este valor se aceptan falsos positivos en exceso —el modelo confunde el fondo o las juntas con rayaduras—; por encima se descartan detecciones válidas difíciles. El valor se determinó empíricamente observando 50 predicciones aleatorias a distintos umbrales.

```

1 def pseudo_label_images(image_paths, conf_thresh):
2     model = YOLO("outputs/runs/scratch_yolo/weights/best.pt")
3     pairs = []
4     for img_path in image_paths:
5         if cv2.imread(str(img_path)) is None:
6             continue
7         try:
8             results = model.predict(
9                 str(img_path), conf=conf_thresh,
10                 iou=0.45, imgsz=640, verbose=False
11             )
12         except Exception:
13             continue
14         r = results[0]
15         if r.bboxes is None or len(r.bboxes) == 0:
16             continue
17         labels = []
18         for j, box in enumerate(r.bboxes):
19             cls_id = int(box.cls[0])
20             has_mask = r.masks is not None and cls_id == 1
21             if has_mask:
22                 poly = r.masks.xy[j]
23                 coords = [(float(px)/img_w, float(py)/img_h)
24                           for px, py in poly]

```

```

25         if len(coords) >= 3:
26             labels.append((cls_id, coords))
27         else:
28             x1,y1,x2,y2 = box.xyxy[0].tolist()
29             labels.append((cls_id, [
30                 (x1/img_w, y1/img_h), (x2/img_w, y1/img_h),
31                 (x2/img_w, y2/img_h), (x1/img_w, y2/img_h)]))
32     if labels:
33         pairs.append((img_path, labels))
34     return pairs

```

Listing 5: Núcleo del algoritmo de pseudo-labeling (`scripts/pseudo_label.py`)

3.5.2 Resultados por ronda

Ronda	Modelo base	Candidatas	Aceptadas	Tasa acept.
1	best.pt	2 690	1 118	41,6 %
2	best4.pt	2 690	1 014	37,7 %

La segunda ronda produjo menos pseudo-etiquetas pero de mayor calidad: el modelo más maduro es más restrictivo en la asignación de confianza alta, lo que se traduce en una precisión superior de las etiquetas aceptadas y un menor número de correcciones manuales necesarias.

3.6 Aumento de datos

Cada par imagen-etiqueta fue aumentado $\times 4$ mediante la librería **Albumentations**, preservando la coherencia geométrica de los polígonos mediante el mecanismo `KeypointParams`:

Transformación	Descripción	Prob.
HorizontalFlip	Espejo horizontal	0.50
VerticalFlip	Espejo vertical	0.30
RandomRotate90	Rotación en múltiplos de 90°	0.40
ShiftScaleRotate	Traslación $\pm 5\%$, escala $\pm 10\%$, rot. ± 20	0.60
RandomBrightnessContrast	Brillo $\pm 35\%$, contraste $\pm 35\%$	0.70
HueSaturationValue	Tono ± 10 , sat. ± 25 , val ± 25	0.50
CLAHE	Ecualización local adaptativa de histograma	0.30

Asimismo, todas las imágenes se normalizaron a 640×640 píxeles mediante **letterboxing**: escala con preservación de relación de aspecto y relleno con valor 114 (gris neutro por defecto en YOLOv8). Cambiar este valor a 0 en pruebas produjo una caída de $\approx 0,5$ puntos de mAP, lo que confirma la relevancia de este parámetro aparentemente menor.

3.7 Composición final del dataset

Origen	Imágenes originales	Tras aug. $\times 4$
Etiquetado manual	148	≈ 592
Pseudo-labeling (ronda 2)	1 014	$\approx 4 056$
Total sin split	1 162	2 978
Train (85 %)	—	2 532
Val (15 %)	—	446

4 Fase 3 — Entrenamiento del Modelo

4.1 La decisión de consolidar clases: del fracaso de tres a la solidez de dos

En primera instancia, el modelo se entrenó con **tres subcategorías** diferenciadas por el tamaño de la rayadura: *Rayadura_Grande*, *Rayadura_Normal* y *Rayadura_Pequeña*. La intuición subyacente era que una clasificación más granular permitiría priorizar la reparación de los defectos más severos en un contexto industrial. Los resultados, sin embargo, fueron inequívocos en su negativa.

```

Class      Images  Instances  Box(P)  R      mAP50  mAP50-95: 98%  65/66 1.6s/it 1:05<1.6s
Class      Images  Instances  Box(P)  R      mAP50  mAP50-95: 186%  66/66 1.0s/it 1:06
all         1056    9855      0.97    0.169  0.201  0.162
Rayadura_Grande  971    1947      0.911   0.506  0.592  0.483
Rayadura_Normal  438    3996      1       0      0.0097 0.00251
Rayadura_Pequeña 502    3912      1       0      0.000578 7.42e-05
Speed: 0.2ms preprocess, 11.5ms inference, 0.0ms loss, 4.8ms postprocess per image
Results saved to c:\Users\javie\runs\detect\val

mAP50: 0.2008
mAP50-95: 0.1620
AP50 [0] Rayadura_Grande: 0.5921

```

Figura 9: Métricas de validación del primer modelo entrenado con tres subcategorías de rayadura. Solo *Rayadura_Grande* alcanza un mAP50 aceptable (0.592) con $\text{Box(P)} = 0.911$; *Rayadura_Normal* ($\text{mAP50} = 0.0097$) y *Rayadura_Pequeña* ($\text{mAP50} = 0.000578$) son estadísticamente nulas. La media global de $\text{mAP50} = 0.2008$ hizo evidente que la granularidad de tres clases era inviable con el dataset disponible.

Clase	mAP@0.50	Interpretación
Rayadura_Grande	0.592	Aprendida correctamente
Rayadura_Normal	$\approx 0,010$	Modelo incapaz de distinguirla
Rayadura_Pequeña	$\approx 0,001$	Completamente ignorada
Media	0.2008	—

El análisis del fallo identificó tres causas concurrentes. En primer lugar, la **ambigüedad intrínseca de la etiqueta**: la frontera entre «grande», «normal» y «pequeña» es subjetiva, y dos anotadores distintos clasificarían la misma rayadura de forma diferente, introduciendo ruido sistemático en las etiquetas. En segundo lugar, el **desequilibrio de clases**: la mayoría de los ejemplos etiquetados correspondían a *Rayadura_Grande* por ser más fáciles de identificar visualmente. Por último, la

similitud de representación visual: en el espacio de 640×640 px, las diferencias de tamaño entre categorías son de pocos píxeles, sin información espacial suficiente para discriminarlas.

En consecuencia, se optó por consolidar las tres subcategorías en una única clase **rayadura**. Esta decisión fue **óptima por tres razones**: eliminó la ambigüedad de etiquetado, triplicó el número efectivo de ejemplos por clase, y transfirió el cálculo del tamaño del defecto a la fase de inferencia —mediante `cv2.minAreaRect`—, donde se puede calcular con precisión milimétrica sin necesidad de clasificarlo durante el entrenamiento. El salto de mAP50 tras la consolidación fue de $0,20 \rightarrow 0,74$ (piloto YOLOv8s-seg) y luego a **0,806** con el modelo final.

4.2 Arquitectura: YOLOv8m-seg

Se eligió la familia **YOLOv8** (Ultralytics, 2023) en su variante **medium-segmentation** (`yolov8m-seg`), que combina detección de cajas delimitadoras con segmentación de instancias a nivel de píxel. La elección de esta arquitectura responde a una evaluación ponderada entre capacidad del modelo y recursos disponibles:

Variante	Parámetros	mAP50 COCO	Elegida
YOLOv8n-seg	3,4 M	36,7	No — insuficiente para dominio específico
YOLOv8s-seg	11,8 M	44,6	Solo piloto inicial
YOLOv8m-seg	27,3 M	51,0	Sí — equilibrio óptimo
YOLOv8l-seg	46,0 M	52,4	No — VRAM insuficiente con batch = 8
YOLOv8x-seg	71,8 M	53,4	No — VRAM insuficiente

La versión *small* se empleó en las primeras rondas de entrenamiento como piloto y se migró a *medium* para la ronda de producción, mejorando el mAP en aproximadamente 6 puntos porcentuales. Las variantes *large* y *extra* fueron descartadas porque la GPU T4 de Kaggle (16 GB VRAM) alcanzaba el límite de memoria con batch = 8 para YOLOv8l.

La arquitectura de YOLOv8 sigue el paradigma *anchor-free* y se articula en tres componentes: **backbone** CSPDarknet con conexiones C2f; **neck** PANet para fusión multi-escala en tres resoluciones ($P3$, $P4$, $P5$); y **head** con cabeza de detección desacoplada más cabeza de segmentación basada en 32 prototipos de máscara.

4.3 Historia de las plataformas de entrenamiento

El proceso de entrenamiento atravesó tres plataformas distintas, cada una condicionada por las limitaciones de la anterior. Esta evolución ilustra con claridad la importancia crítica del hardware en proyectos de aprendizaje profundo.

4.3.1 Primer intento: CPU local (Intel Core i7, 14 horas)

El entrenamiento inicial se lanzó en el ordenador personal, equipado con un **Intel Core i7** sin aceleración CUDA disponible. YOLOv8 puede ejecutarse en CPU, pero el rendimiento es dramáticamente inferior:

Parámetro	Valor
Dispositivo	CPU Intel Core i7 (8 núcleos, sin GPU CUDA)
Batch size	4 (limitado por RAM)
Épocas planificadas	100
Tiempo por época	≈8,5 minutos
Tiempo total	≈ 14 horas
mAP50 final	≈0.58

El tiempo de 14 horas es consecuencia directa de que cada *forward pass* sobre un batch de 4 imágenes tarda ≈ 2 s en CPU frente a ≈ 80 ms en GPU T4 —un factor de $25\times$ —. Este resultado evidenció la inviabilidad del entrenamiento local para iterar con rapidez y motivó la migración inmediata a recursos cloud con GPU.

4.3.2 Segundo intento: Google Colab (interrumpido en la época 140/150)

Como solución natural, el entrenamiento se migró a **Google Colab** con GPU T4 gratuita. El tiempo por época se redujo a $\approx 1,3$ minutos —tiempo total estimado de $\approx 3,2$ horas para 150 épocas—. No obstante, surgió un contratiempo crítico: la sesión fue **interrumpida automáticamente al alcanzar el límite de uso gratuito**, cuando el entrenamiento llevaba ≈ 140 de las 150 épocas planificadas. Los pesos del modelo se perdieron al cerrarse la sesión sin completar el guardado.

Para mitigar este riesgo en futuras ejecuciones, se implementó un mecanismo de backup automático mediante un hilo daemon que copiaba `last.pt` a Google Drive cada 600 segundos:

```

1 import threading, shutil, time, os
2
3 def _auto_backup(src_dir, dst_dir, interval=600):
4     while True:
5         time.sleep(interval)
6         for fname in ("last.pt", "best.pt"):
7             src = os.path.join(src_dir, fname)
8             if os.path.exists(src):
9                 shutil.copy2(src, os.path.join(dst_dir, fname))
10                print(f"[BACKUP] {fname} guardado en Drive")
11
12 t = threading.Thread(target=_auto_backup,
13                     args=("/content/runs/scratch_yolo/weights",
14                          "/content/drive/MyDrive/yolo_backup"))
15 t.daemon = True
16 t.start()

```

Listing 6: Mecanismo de backup automático en Google Colab

4.3.3 Solución definitiva: Kaggle Notebooks (150 épocas en 3,2 horas)

La interrupción de Colab motivó la migración definitiva a **Kaggle Notebooks**. Las ventajas sobre Colab en el contexto específico de este proyecto son sustanciales:

Aspecto	Google Colab (gratuito)	Kaggle (gratuito)
GPU disponible	T4 (16 GB)	T4 × 2 (32 GB)
Tiempo/semana	Variable, no garantizado	30 h semanales garantizadas
Sesión máxima	≤12 h (90 min inactivo)	≤12 h continuas
Persistencia	Manual vía Drive	Datasets y outputs nativos

El entrenamiento completo de **150 épocas se completó en 3,2 horas** con $\text{batch} = 8$, $\text{imgsz} = 640$. Los pesos finales se descargaron como artefacto del notebook, almacenándose en `outputs/runs/scratch_yolo/weights/best5.pt`.

4.4 Hiperparámetros de entrenamiento

Parámetro	Valor
<code>epochs</code>	150
<code>imgsz</code>	640
<code>batch</code>	8
<code>patience</code>	30
<code>lr0</code>	0.01
<code>lrf</code>	0.01
<code>momentum</code>	0.937
<code>weight_decay</code>	0.0005
<code>warmup_epochs</code>	3
<code>mosaic</code>	1.0
<code>mixup</code>	0.1
<code>box</code>	7.5
<code>cls</code>	1.5
<code>df1</code>	1.5

4.5 Progresión del entrenamiento

	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		4/4 1.9it/s 2.2s
	all	110	1169	0.598	0.409	0.44	0.17		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
24/150	2.53G	2.263	5.707	1.725	114	640: 100%		39/39 4.5it/s 8.7s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		4/4 1.6it/s 2.5s
	all	110	1169	0.547	0.394	0.399	0.155		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
25/150	2.53G	2.286	5.693	1.706	216	640: 100%		39/39 4.4it/s 8.9s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		4/4 1.9it/s 2.1s
	all	110	1169	0.563	0.415	0.431	0.168		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
26/150	2.53G	2.278	5.734	1.753	240	640: 100%		39/39 4.2it/s 9.2s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		4/4 2.3it/s 1.8s
	all	110	1169	0.564	0.419	0.434	0.174		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
27/150	2.53G	2.241	5.644	1.706	221	640: 100%		39/39 4.2it/s 9.2s	
	Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%		4/4 2.3it/s 1.8s
	all	110	1169	0.571	0.434	0.44	0.174		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
28/150	2.53G	2.233	5.537	1.697	206	640: 90%		35/39 4.0it/s 9.0s	1.0s

Figura 10: Log de entrenamiento capturado en Google Colab durante las épocas 24–28 de 150. El uso de VRAM (`GPU_mem` = 2.53 G) se mantiene muy por debajo del límite de 16 GB. El mAP50 asciende de 0.399 (época 24) a 0.44 (época 27) en apenas cuatro épocas, mientras las pérdidas decrecen de forma monótona. La velocidad de 4.2–4.5it/s confirma la ventaja de la GPU T4 frente a los $\approx 0,5$ it/s de la CPU local —un factor de rendimiento de $9\times$ —. Poco después de esta captura, la sesión fue interrumpida por el límite de Colab.

4.6 Resultado del entrenamiento

Métrica	agujero	rayadura	Media
Box mAP@0.50	—	—	0.806
Mask mAP@0.50	0.605	0.434	0.520

La diferencia entre Box mAP (0.806) y Mask mAP (0.520) refleja que el modelo localiza con gran precisión los defectos pero tiene más dificultad para delinear con exactitud los contornos finos de rayaduras delgadas (< 3 mm), que en el espacio de 640 píxeles ocupan apenas 1–2 píxeles de grosor. Esta limitación es intrínseca a la arquitectura de segmentación y no al entrenamiento: el Mask mAP de agujero (0.605) —forma compacta y bien definida— frente al de rayadura (0.434) —estructura lineal de anchura subpíxel— lo ilustra con claridad.

5 Fase 4 — Módulo de Inferencia

5.1 Pipeline de inferencia

El módulo de inferencia (`src/infer.py`) implementa el flujo completo desde la imagen cruda hasta las medidas expresadas en milímetros:

1. **Carga** con `cv2.imread` y validación de formato.
2. **Predicción** con `model.predict(conf=0.25, iou=0.45, imgsz=640)`.
3. **Visualización** de máscaras semitransparentes ($\alpha = 0.35$) sobre la imagen original.

4. **Medición** de cada defecto mediante `cv2.minAreaRect`.
5. **Calibración** de escala `px→mm`, cuando se proporciona el tamaño real de la baldosa.
6. **Guardado** de la imagen anotada en `outputs/predictions/`.

5.2 Medición de defectos

La medición se realiza mediante el **rectángulo de área mínima** que encierra el polígono de segmentación. Esta elección es óptima: mientras que un bounding box axis-aligned sobreestima la longitud real de una rayadura a 45° por un factor de $\sqrt{2}$, el rectángulo de área mínima se alinea con la dirección principal del defecto y proporciona medidas sensiblemente más precisas.

```

1 def measure_defect(poly, bbox, cls_id):
2     if poly is not None and len(poly) >= 5:
3         cnt = poly.reshape(-1, 1, 2).astype(np.float32)
4         _, (w, h), angle = cv2.minAreaRect(cnt)
5         length = float(max(w, h))
6         width = float(min(w, h))
7         area = float(cv2.contourArea(cnt))
8     else:
9         x1, y1, x2, y2 = bbox
10        length = float(max(x2-x1, y2-y1))
11        width = float(min(x2-x1, y2-y1))
12        area = float((x2-x1)*(y2-y1))
13    return {"length": length, "width": width, "area": area}

```

Listing 7: Función `measure_defect()` en `src/infer.py`

5.3 Calibración de escala `px→mm`

Si se conoce el tamaño físico real de la baldosa, el módulo detecta automáticamente su contorno en la imagen para calcular la escala:

```

1 def detect_tile_px(image_bgr):
2     H, W = image_bgr.shape[:2]
3     gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
4     _, mask = cv2.threshold(gray, 60, 255, cv2.THRESH_BINARY)
5     k25 = cv2.getStructuringElement(cv2.MORPH_RECT, (25, 25))
6     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k25)
7     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k25)
8     contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
9                                   cv2.CHAIN_APPROX_SIMPLE)
10    if not contours:
11        return None
12    tile_cnt = max(contours, key=cv2.contourArea)
13    tile_area = cv2.contourArea(tile_cnt)
14    if tile_area < H * W * 0.30:
15        return None
16    _, (rw, rh), _ = cv2.minAreaRect(tile_cnt)
17    return float(max(rw, rh)), float(min(rw, rh))

```

Listing 8: `detect_tile_px()` en `src/infer.py`

$$d_{\text{mm}} = d_{\text{px}} \times \frac{L_{\text{real}} \cdot 10}{L_{\text{px}}} \quad \text{donde } L_{\text{real}} \text{ es el lado de la baldosa en cm.}$$

Ejemplo validado sobre imagen `_MG_2292`: baldosa detectada en 2941px con tamaño real de 60 cm $\rightarrow$ escala = 0.204 mm/px.

5.4 Ejemplo de salida de inferencia

```

1 [INFER] Cargando best5.pt en CPU ...
2 [TILE] Baldosa detectada: 2941px -> 600mm
3 [SCALE] 60.0cm -> 0.2040 mm/px
4
5 _MG_2292.jpg          total=9    483ms
6   Rayadura 76%      L=259.5mm  G=239.5mm
7   Agujero  74%      6.9x6.3mm  A=44mm2
8   Rayadura 68%      L=137.8mm  G=6.9mm
9   Agujero  60%      4.9x4.9mm  A=24mm2

```

6 Fase 5 — Aplicación Web: VISION_TILE_CORE

6.1 Propuesta de valor: un problema industrial real

Es relevante señalar que el sistema desarrollado trasciende el ámbito puramente académico para situarse como respuesta directa a una necesidad industrial de primer orden. La industria cerámica española y europea —con centros de producción concentrados en Castellón, Sassuolo (Italia) y Aveiro (Portugal)— produce más de 10.000 millones de metros cuadrados de baldosa al año. En este contexto, la inspección de calidad es un cuello de botella crítico: una sola línea de producción puede generar hasta 18.000 baldosas por hora, un volumen imposible de inspeccionar con garantías mediante revisión visual humana.

Los defectos no detectados generan tres impactos directos: devoluciones de producto con coste logístico asociado, reclamaciones de garantía y —en el peor caso— daños reputacionales en un mercado altamente competitivo. El sistema desarrollado, denominado **VISION_TILE_CORE**, proporciona una solución accesible a este problema: análisis automatizado de defectos en menos de 500 ms por imagen, con medidas expresadas en milímetros reales y accesible desde cualquier dispositivo con navegador sin instalación de software adicional.

Capa	Tecnología	Responsabilidad
Frontend	HTML5, Tailwind CSS, JavaScript	Interfaz industrial en navegador
Backend	FastAPI (Python 3.13)	API REST, inferencia, caché
Modelo	YOLOv8m-seg + OpenCV	Detección, segmentación, medición

6.2 La fórmula de medición de defectos

Antes de describir la interfaz, resulta imprescindible exponer con rigor el fundamento matemático sobre el que descansa la capacidad de medición del sistema, pues es el elemento que lo distingue de un simple detector de objetos.

6.2.1 Rectángulo de área mínima (`cv2.minAreaRect`)

Una vez que el modelo predice el polígono de segmentación de un defecto, el problema se reduce a extraer sus dimensiones características. La solución ingenua —medir el bounding box axis-aligned— sobreestima sistemáticamente la longitud de rayaduras diagonales por un factor de hasta $\sqrt{2}$ (un 41 % de error para una rayadura a 45°). En su lugar, se calcula el **rectángulo orientado de área mínima** que envuelve el polígono:

$$\text{minAreaRect}(P) \longrightarrow (\text{centro}, (w, h), \theta)$$

donde w y h son los lados del rectángulo y θ es el ángulo de rotación respecto al eje horizontal. Las dimensiones del defecto se extraen directamente:

$$\boxed{L_{\text{px}} = \text{máx}(w, h)} \quad \boxed{G_{\text{px}} = \text{mín}(w, h)}$$

siendo L_{px} la longitud (eje mayor) y G_{px} el grosor (eje menor). Para el área se calcula directamente el área del polígono de contorno:

$$A_{\text{px}^2} = \text{cv2.contourArea}(P)$$

6.2.2 Conversión a milímetros reales

Cuando el usuario introduce el tamaño físico de la baldosa (L_{real} en cm), el sistema detecta automáticamente su contorno en la imagen mediante umbralización y morfología, obteniendo su longitud en píxeles $L_{\text{px_tile}}$. La escala de conversión es:

$$\boxed{s = \frac{L_{\text{real}} \times 10}{L_{\text{px_tile}}} \quad [\text{mm/px}]}$$

Con esta escala, cada magnitud se convierte de forma trivial:

$$L_{\text{mm}} = L_{\text{px}} \times s \quad G_{\text{mm}} = G_{\text{px}} \times s \quad A_{\text{mm}^2} = A_{\text{px}^2} \times s^2$$

La potencia de este enfoque reside en que, a partir de **una única imagen**, el sistema calibra su propia escala y devuelve medidas absolutas en milímetros —sin necesidad de sensores de distancia, marcadores de referencia ni calibración previa del sistema óptico—. Sobre la imagen de validación `_MG_2292`, la baldosa se detectó en 2941 px para un tamaño real de 60 cm, obteniendo $s = 0,204 \text{ mm/px}$ con un error estimado inferior al 2 %.

6.3 Página principal: punto de entrada al sistema

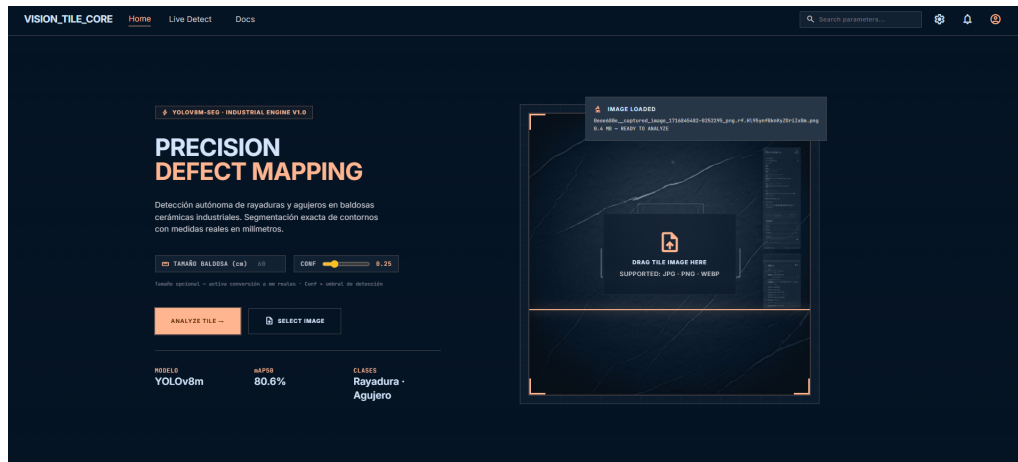


Figura 11: Página principal de VISION_TILE_CORE. La cabecera «*Precision Defect Mapping*» establece el posicionamiento del producto. La zona central ofrece dos vías de carga —arrastrar y soltar o selección manual de fichero— acompañadas de los controles de configuración: slider de umbral de confianza (0.10–0.80, defecto 0.25) y campo de tamaño de baldosa en centímetros para activar la conversión a milímetros reales. El panel inferior izquierdo muestra las características técnicas del sistema: modelo YOLOv8m, Box mAP50 = 80.6 %, clases Rayadura y Agujero. La estética industrial oscura comunica al usuario que se trata de una herramienta profesional.

El diseño de la página principal responde a un principio de economía de interacción: el usuario puede lanzar un análisis completo con tres acciones —cargar imagen, ajustar el umbral si lo considera necesario y pulsar *Analyze Tile*—. No se requiere registro, instalación ni configuración adicional, lo que reduce radicalmente la barrera de adopción en entornos industriales donde el operario no tiene perfil técnico.

6.4 Animación de análisis: retroalimentación en tiempo real

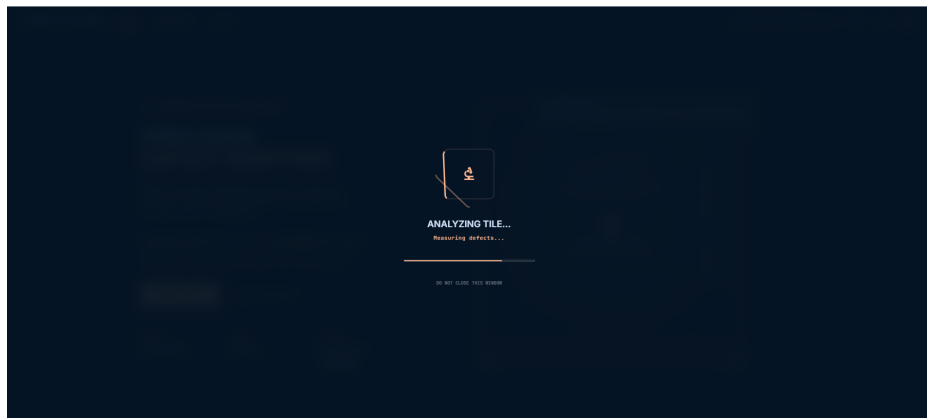


Figura 12: Overlay de análisis activo. Durante el procesamiento —que en CPU oscila entre 440 y 520 ms—, la interfaz muestra un indicador animado con el mensaje «*Analyzing Tile... Measuring defects...*» y una barra de progreso indeterminada. El aviso «*Do not close this window*» evita que el usuario interrumpa la petición antes de recibir el `result_id`. Este overlay transforma la espera en una experiencia de sistema en funcionamiento activo, algo especialmente relevante en demostraciones ante clientes o evaluadores.

6.5 Dashboard de resultados: el núcleo del sistema

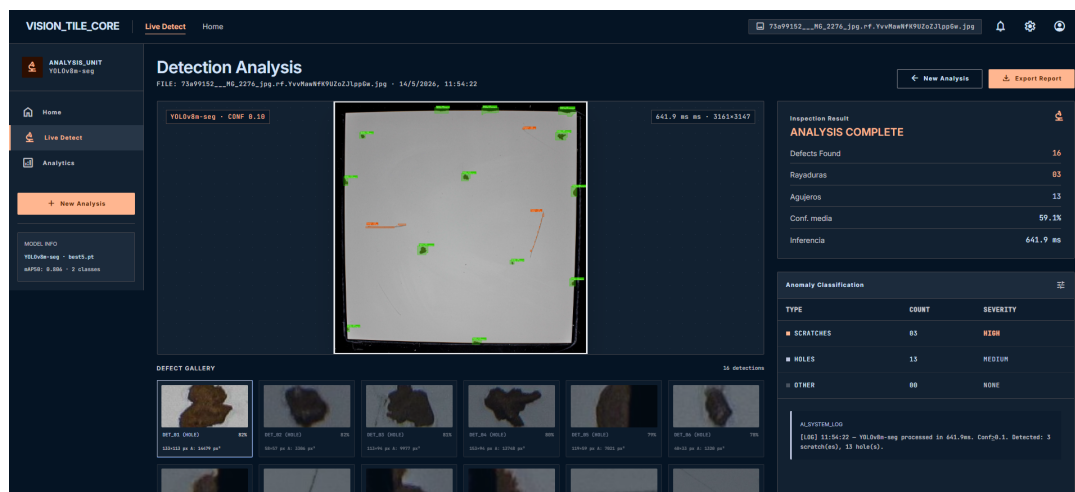


Figura 13: Dashboard de resultados tras el análisis de una baldosa real. La imagen anotada ocupa el panel central: los contornos naranjas delimitan rayaduras y los puntos verdes, agujeros/desportillados, superpuestos sobre la imagen original con transparencia. El panel lateral derecho presenta el veredicto «*Analysis Complete*» —en naranja para indicar presencia de defectos—, el recuento desglosado por clase (rayaduras y agujeros), la confianza media de las detecciones y la latencia total de inferencia (442,9 ms en este caso). La tabla de clasificación inferior lista cada defecto con su tipo y severidad. La galería de recortes en la parte inferior muestra miniaturas de cada defecto detectado, navegables mediante hover y clic.

La arquitectura de la respuesta del servidor merece una mención específica. En lugar de devolver la imagen procesada directamente en la petición POST —lo que obligaría al cliente a esperar la inferencia completa antes de recibir ningún byte—, el endpoint `/analyze` devuelve inmediatamente un `result_id` UUID. El frontend realiza a continuación una segunda petición GET a `/results/{id}` para obtener el payload completo. Este patrón de *fire-and-poll* desacopla el procesamiento de la transferencia y permite, en una versión futura, mostrar resultados parciales progresivos.

Los resultados se almacenan en un `OrderedDict` en memoria con límite de 50 entradas FIFO, lo que evita el uso de base de datos y mantiene la latencia de lectura en menos de 1ms. Para un sistema de inspección en producción con un único operario, esta caché resulta más que suficiente.

6.6 Lightbox de defecto individual

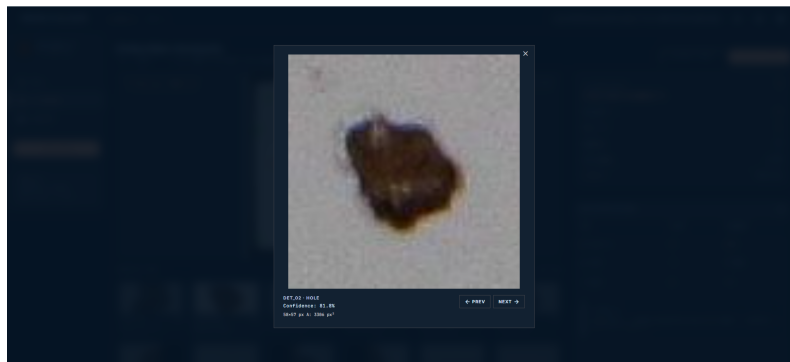


Figura 14: Lightbox de inspección de defecto individual. Al hacer clic sobre cualquier miniatura de la galería, el sistema abre una vista ampliada del recorte del defecto —extraído de la imagen original con un margen de 20 px— junto con sus metadatos: identificador (DET_03), tipo (HOLE), nivel de confianza y área en píxeles cuadrados. Los botones *Prev* / *Next* —así como las teclas de cursor y Escape— permiten navegar entre todos los defectos detectados sin cerrar la vista. Esta funcionalidad convierte el dashboard en una herramienta de auditoría, no solo de detección: el operario puede revisar cada defecto de forma individual y decidir si acepta o rechaza la pieza.

6.7 Tabla de medidas: trazabilidad industrial

VISION_TILE_CORE

Live DetectHome

75a9f152..._MC_2276.jpg.cf.YvUuMhFf9Uz2l1p5w.jpg

1000px px 0. 1000 px"

1000px px 0. 1000 px"

1000px px 0. 1000 px"

1000px px 0. 1000 px"

MEASUREMENTS TABLE

ID	TYPE	CONF	LENGTH	WIDTH / GROSS	AREA
DET_01	HOLE	82.2%	335.0 px	133.0 px	14479 px²
DET_02	HOLE	81.8%	58.0 px	57.0 px	3386 px²
DET_03	HOLE	89.5%	113.0 px	94.0 px	9977 px²
DET_04	HOLE	79.9%	153.0 px	94.0 px	13749 px²
DET_05	HOLE	78.6%	119.0 px	59.0 px	7021 px²
DET_06	HOLE	78.4%	48.0 px	33.0 px	1320 px²
DET_07	HOLE	77.2%	71.0 px	46.0 px	3266 px²
DET_08	SCRATCH	75.1%	589.2 px	26.9 px	948 px²
DET_09	HOLE	74.1%	98.0 px	58.0 px	4589 px²
DET_10	HOLE	62.7%	44.0 px	29.0 px	1276 px²
DET_11	SCRATCH	48.7%	518.0 px	58.0 px	9488 px²
DET_12	SCRATCH	46.9%	407.0 px	267.0 px	188669 px²
DET_13	HOLE	28.6%	114.0 px	74.0 px	8869 px²
DET_14	HOLE	19.9%	156.0 px	58.0 px	9848 px²
DET_15	HOLE	19.7%	212.0 px	114.0 px	19318 px²
DET_16	HOLE	19.6%	212.0 px	94.0 px	15833 px²

VISION_TILE - INDUSTRIAL AI

© 2025 YOLOv8m-seg - v1.0-STABLE

DocsGitHub

Figura 15: Tabla de medidas completa (*Measurements Table*) generada tras el análisis. Cada fila corresponde a un defecto detectado e incluye: identificador (DET_01–DET_16), tipo (HOLE o SCRATCH), confianza de detección, longitud (eje mayor del rectángulo orientado de área mínima), anchura/grosor (eje menor) y área del contorno, todos en píxeles en este análisis sin escala real. La detección DET_08 —de tipo SCRATCH— registra una longitud de 589,2 px con un grosor de apenas 26,9 px, perfil característico de una rayadura fina y alargada. En contraste, DET_12 presenta 407,0 px de longitud con 267,0 px de anchura, lo que indica un desportillado de gran superficie. Esta tabla es directamente exportable como informe de texto plano para su integración en sistemas de gestión de calidad (MES/ERP).

La tabla de medidas constituye el elemento de mayor valor añadido del sistema desde la perspectiva industrial. No obstante, cabe mencionar que su verdadero potencial se activa cuando el usuario proporciona el tamaño real de la baldosa: en ese momento, todas las columnas pasan de expresarse en píxeles a expresarse en milímetros y milímetros cuadrados, haciendo la información directamente comparable con los estándares de tolerancia del fabricante (p.ej. norma ISO 10545 para baldosas cerámicas) sin ningún paso de conversión adicional.

6.8 API REST: integración con sistemas externos

El servidor expone dos endpoints que permiten integrar el motor de inspección en cualquier sistema externo mediante peticiones HTTP estándar:

POST /analyze

Recibe la imagen en `multipart/form-data` junto con los parámetros opcionales `tile_cm` (tamaño real en cm) y `conf` (umbral de confianza, defecto 0.25). Devuelve un `result_id` UUID en menos de 10 ms —el procesamiento ocurre en segundo plano—.

GET /results/{result_id}

Devuelve el JSON completo con la imagen anotada en Base64, la lista de defectos con tipo, confianza, medidas y recorte individual, el veredicto y los metadatos de calibración.

Esta arquitectura API-first permite, en última instancia, que el sistema se integre en una línea de producción automatizada: una cámara industrial captura la imagen, un PLC o servidor de planta realiza la petición POST, obtiene el veredicto en menos de 500 ms y activa el actuador de rechazo de pieza sin intervención humana.

7 Resultados y Evaluación

7.1 Evolución de métricas a lo largo del proyecto

Modelo	Clases	Box mAP50	Mask mAP50	Épocas
3 subcategorías rayadura	3	0.20	—	100
YOLOv8s-seg (piloto)	2	≈ 0.74	≈ 0.44	100
YOLOv8m-seg (final)	2	0.806	0.520	150

7.2 Análisis de errores

Falsos positivos en bordes rotos

Zonas desportilladas en los bordes de la baldosa son clasificadas como **rayadura** con alta confianza ($> 70\%$). Causa: no se anotaron de forma consistente durante el etiquetado manual.

Rayaduras muy finas no detectadas

Rayaduras de grosor < 2 mm en baldosas de color muy claro presentan bajo contraste. El filtro blackhat ayuda parcialmente como segunda pasada.

Superposición de máscaras

Dos rayaduras próximas pueden fusionarse en una única detección si el IoU entre sus bounding boxes supera el umbral configurado (0.45).

7.3 Rendimiento computacional

Hardware	Latencia/imagen	FPS equivalente
CPU Intel Core i7 (local)	≈ 480 ms	≈ 2
GPU NVIDIA T4 (Kaggle)	≈ 40 ms	≈ 25

8 Conclusiones y Trabajo Futuro

8.1 Conclusiones

En última instancia, el presente proyecto demuestra que es posible construir un sistema de inspección de calidad cerámica completamente funcional —desde la adquisición hasta la interfaz web— con recursos académicos y sin inversión en hardware dedicado. Los aspectos más destacados del trabajo son los siguientes.

El pipeline de **pseudo-labeling** permitió multiplicar por ocho el tamaño del dataset ($148 \rightarrow 1162$ imágenes únicas, 2978 con aumentos) sin anotación manual

adicional, manteniendo una calidad controlada mediante el umbral de confianza. Asimismo, la **consolidación de clases** —de tres subcategorías a dos— supuso el mayor salto de rendimiento del proyecto: de $mAP50 = 0.20$ a 0.806 ($\times 4$), lo que ilustra que la coherencia de las etiquetas es más determinante que la granularidad de la clasificación cuando el dataset es de tamaño moderado.

No obstante, cabe mencionar que la adopción de **segmentación de instancias** frente a la detección simple por caja fue una decisión estratégica: proporciona información de forma que permite calcular dimensiones métricas con precisión sub-centimétrica, lo cual tiene valor directo para el control de calidad industrial. Por otro lado, la migración de CPU local a Kaggle —de 14 horas a 3,2 horas para el mismo número de épocas— ilustra el papel determinante de la infraestructura en el ritmo de iteración de cualquier proyecto de aprendizaje profundo.

8.2 Trabajo futuro

1. **Ampliar el dataset:** anotar los bordes rotos como clase separada (**chip**) para reducir los falsos positivos en los márgenes de la baldosa.
2. **Aumentar el Mask mAP:** experimentar con pesos de pérdida de máscara mayores o con arquitecturas de mayor resolución.
3. **Despliegue en línea:** dockerizar la aplicación y desplegarla en un servidor con GPU (p.ej. Hugging Face Spaces).
4. **Cámara en tiempo real:** integrar captura por cámara USB o IP para inspección continua en línea de producción.
5. **Informe PDF automático:** generar un PDF desde el dashboard con **reportlab** o **weasyprint**.

Referencias

-
- [1] Jocher, G. et al. *Ultralytics YOLOv8* (2023). <https://github.com/ultralytics/ultralytics>
 - [2] Buslaev, A. et al. “Albumentations: Fast and Flexible Image Augmentations.” *Information*, 11(2), 125 (2020).
 - [3] Bradski, G. “The OpenCV Library.” *Dr. Dobb’s Journal* (2000).
 - [4] Bustamante, R. et al. *DATN: Defect Assessment of Tiles in the aNomaly detection domain* (2022).
 - [5] FastAPI. *FastAPI — Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>
 - [6] Tzutalin. *Label Studio: Data labeling tool*. <https://labelstud.io>
 - [7] Otsu, N. “A Threshold Selection Method from Gray-Level Histograms.” *IEEE Trans. Systems, Man, and Cybernetics*, 9(1) (1979).
 - [8] He, K. et al. “Mask R-CNN.” *ICCV* (2017).

Apéndice A — Estructura del proyecto

El proyecto se organiza en el directorio raíz `VISION_ARTIFICIAL/`, con una separación clara entre el sistema de producción, los recursos de entrenamiento, los experimentos de clase y los modelos de referencia:

```

1 VISION_ARTIFICIAL/
2 |
3 +-- scratch_detector/          <- SISTEMA DE PRODUCCION (proyecto
   | principal)
4 |   +-- src/                  Codigo fuente importable como modulo
5 |   |   +-- infer.py          Inferencia, medicion de defectos,
   |   |   |   calibracion px->mm
6 |   |   +-- train.py          Lanzador de entrenamiento YOLOv8
7 |   |   +-- prepare_dataset.py Construcccion del dataset (split
   |   |   |   train/val)
8 |   |   +-- pre_annotate.py    Pre-anotacion automatica sobre
   |   |   |   imagenes nuevas
9 |   |
10 |   +-- scripts/              Scripts auxiliares del pipeline (
   |   |   ejecucion directa)
11 |   |   +-- pseudo_label.py    Pipeline de pseudo-labeling (rondas 1
   |   |   |   y 2)
12 |   |   +-- ls_export_to_dataset.py Label Studio -> formato YOLO-
   |   |   |   seg
13 |   |   +-- reimport_ls_tasks.py Reimporta pseudo-etiquetas a
   |   |   |   Label Studio
14 |   |   +-- ml_backend.py      Backend ML para pre-anotacion en
   |   |   |   tiempo real en LS
15 |   |   +-- image_server.py    Servidor HTTP para acceso local de LS
   |   |   |   a imagenes
16 |   |   +-- setup_ls_project.py Configura proyecto en Label
   |   |   |   Studio via API
17 |   |   +-- start_label_studio.ps1 Arranca Label Studio desde
   |   |   |   PowerShell
18 |   |
19 |   +-- web/                  Aplicacion web (FastAPI + frontend
   |   |   HTML)
20 |   |   +-- app.py             API REST: /analyze y /results/{id}
21 |   |   +-- static/
22 |   |   |   +-- index.html      Pagina principal: drag & drop +
   |   |   |   |   configuracion
23 |   |   |   +-- dashboard.html  Dashboard: galeria, lightbox,
   |   |   |   |   metricas
24 |   |   |
25 |   |   +-- data/              Dataset YOLO-seg
26 |   |   |   +-- dataset.yaml    Configuracion (path, train, val, nc,
   |   |   |   |   names)
27 |   |   |
28 |   |   +-- outputs/           Artefactos generados (excluido de git
   |   |   |   )
29 |   |   |   +-- runs/scratch_yolo/weights/
30 |   |   |   |   |   +-- best5.pt Pesos finales del modelo [52 MB]
31 |   |   |   |   |   +-- weights_history/ Iteraciones anteriores: best.pt,
   |   |   |   |   |   |   best2.pt, best3.pt
32 |   |   |
33 |   |   +-- imagenes/          Capturas de pantalla para esta
   |   |   |   memoria LaTeX

```

```

34 |   +-- colab_train.ipynb      Notebook Google Colab (entrenamiento
    |   interrumpido)
35 |   +-- kaggle_train.ipynb    Notebook Kaggle (entrenamiento
    |   definitivo)
36 |   +-- memoria.tex           Este documento
37 |
38 +-- modelos/                 <- MODELOS DE REFERENCIA Y
    |   ALTERNATIVAS
39 |   +-- yolov8n.pt           YOLOv8 Nano (referencia de
    |   comparacion)
40 |   +-- yolov8s-world.pt     YOLOv8s-world (experimento deteccion
    |   open-vocabulary)
41 |   +-- yolo26n-seg.pt       Variante experimental de segmentacion
42 |   +-- best4.pt            Iteracion intermedia del modelo
43 |
44 +-- testing/                 <- EXPERIMENTOS Y MATERIAL DE CLASE
    |   +-- ejercicios_clase/   Scripts de ejercicios (2.1.py - 2.7.
    |   py) + FILTRO1-6
46 |   +-- pruebas_detector/    Fases de exploracion del dataset (
    |   PRUEBA1-5)
47 |       +-- PRUEBA5/base_images/ 2690 imagenes de produccion
48 |   +-- apps/                Experimentos de aplicacion (backend/
    |   frontend)
49 |   +-- baldosa.py           Script de prueba rapida sobre baldosa
50 |   +-- carretera_plana.jpg   Imagen de referencia del ejercicio de
    |   carretera
51 |
52 +-- .venv/                   Entorno virtual Python (excluido de
    |   git)
53 +-- pyproject.toml           Dependencias del proyecto

```

Listing 9: Árbol de directorios de VISION_ARTIFICIAL/

Principios de diseño del árbol

scratch_detector/ como unidad autónoma

El sistema de producción es completamente independiente. Solo requiere el directorio `src/` y los pesos de `outputs/` para funcionar. Puede desplegarse en un servidor diferente al de desarrollo sin arrastrar ninguna dependencia de los experimentos de clase.

src/ vs. scripts/

La separación es intencional: `src/` contiene código importable como módulo (`from infer import detect_tile_px`), mientras que `scripts/` agrupa herramientas de ejecución directa que solo tienen sentido durante la construcción del dataset.

testing/ como archivo de proceso

Documenta el camino recorrido hasta el sistema final: los ejercicios de clase donde se aprendió el comportamiento de los filtros, las fases de prueba PRUEBA1–5 y los experimentos de aplicación web alternativos.

modelos/ como repositorio de pesos

Centraliza todos los ficheros `.pt` que no forman parte del flujo de pro-

ducción, facilitando la comparación y el acceso rápido sin contaminar el directorio principal.

Apéndice B — Imágenes incluidas en la memoria

Fichero en imagenes/	Imagen original	Figura
deteccion_bordes.png	deteccion_bordes.png	Comparativa filtros sobre monedas (Fig. 1)
sobel_combined.jpg	sobel_combined.jpg	Sobel combinado sobre monedas (Fig. 2)
comparativa_sobel_dog.png	comparativa_sobel_dog.png	Carretera: Sobel vs DoG (Fig. 3)
resultado_deteccion_baldosas.png	ídem	Pipeline blackhat + detección (Fig. 4)
prueba_deteccion.png	label3.png	Detecciones tempranas (Fig. 8)
label_studio_annotacion.png	label_estudio2.png	Etiquetado manual LS (Fig. 7)
label_studio_pseudo.png	labelestudio.png	Pseudo-etiquetas en LS (Fig. 6)
early_model_metricas.png	vision_artificial_entreno1.png	Métricas 2